

[Blog Home](#) > [Development](#) > [Fundamenta...](#)

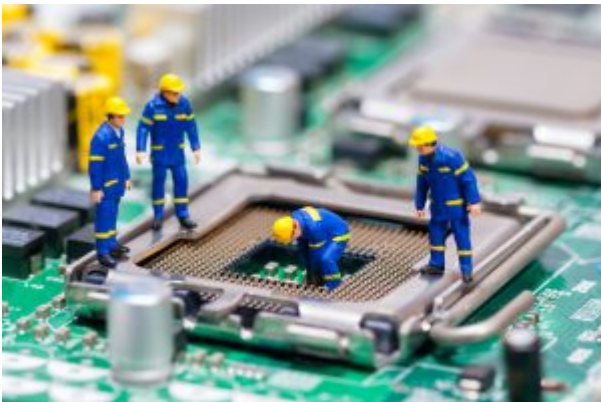
Development

Fundamentals of MIPS Programming in Assembly Language

 Udemy Editor

[Share this article](#) 

“Microprocessor without Interlocked Pipeline Stages” (abbreviated MIPS) is a computer processor architecture developed by MIPS Technologies, and is often used when teaching assembly language programming in computer science courses. The design has also been licensed to manufacturers, such as the Sony Corporation for its early PlayStation range of games consoles and handhelds, and can regularly be found embedded in electronic devices.



Although this introduction explains many of the basic concepts, programming in assembly language requires a good knowledge of microprocessor-based systems. [Computer Science for Everyone with Java at Udemy.com](#) is an excellent resource for learning the principles of how computers work, and acquiring a [basic knowledge of digital electronics, logic, and circuits](#) is always useful before programming in a low-level language like assembly.

Editors and Simulators

Assembly language requires very few pieces of software to learn and use. However, despite its popularity, you may not have access to a machine with a MIPS processor inside and will need a suitable emulator (or simulator) to run the examples described in this article.

Spim is a 32-bit MIPS simulator, and is slightly unusual in that it does not run binary (compiled/assembled) executables. Instead, Spim accepts your assembly language programs in their source code format. Versions of Spim are available for Windows, GNU/Linux, and Mac OS X computers.

Other than that, all you need to begin with is a basic text editor such as Notepad, Vi, or Emacs.

A Program

The example below is a short MIPS assembly language program – it consists of a single instruction and illustrates the basic structure of an assembly language source file.

```
#
# Udemy.com
# MIPS Programming in Assembly Language
#
        .data
var1:   .byte 1           # declare a single byte
var2:   .half 6           # declare a 16-bit halfword
var3:   .word 9           # declare a 32-bit word
str1:   .ascii "Text"     # declare a string of characters
        .space 5          # reserve 5 bytes of space
        .asciiz "Text"    # declare a null-terminated string
        .float 3.14       # declare a 32-bit floating point number
        .double 6.28      # declare a 64-bit floating pointer number
        .text
        lb $8, var1
```

From this, you can see a few of the expectations that most MIPS assemblers and simulators have:

- Comments begin with a hash symbol (#) and continue to the end of the line.
- Variables are declared in the *.data* segment, and assembly language code must be in the *.text* segment.
- Labels work as with in other versions of assembly language.

Declaring User Data (Variables)

Variables are values used by a running program that can be changed at any time. In MIPS assembly language, allocating space for variables must be done in the *.data* segment, and generally requires you to specify the data type to be used.

The example code above shows the eight different types of declaration. It does not show how multiple values of a single type can be inserted. You can do this by separating each value by a comma. For example:

```
vars:    .byte    1,2,3,4    # declares 4 bytes with the values 1-4
```

Many of the data declarations so far have included a label at the start of the line. This is optional, but is useful way of giving a name to the area of memory containing the data so that it can be referenced later in the program. Labels must begin with a letter and be followed by letters, numbers, or the underscore character. The end of the label is marked with a colon, but this is not part of its name.

Top courses in Assembly Language

x86 Assembly Language Programming From...

Israel Gbati
4.5 ★★★★★ (1,403)
Bestseller

Assembly Language Programming for Revers...

Paul Chin
4.7 ★★★★★ (97)
Bestseller

x86 Assembly Language Programming Masters...

Daniel McCarthy
4.5 ★★★★★ (390)

Learn Assembly Language by Making Games for th...

Gustavo Pezzi, Pikuma .com
4.7 ★★★★★ (2,089)
Highest Rated

More Assembly Language Courses

You can use labels in the *.text* segment to create named markers for different parts of the program.

MIPS Registers

MIPS processors have 32 general-purpose registers (numbered 0–31) that are built-in to the chip itself and can be used to hold the results of calculations and operations. They can be accessed using their number – by prefixing a dollar symbol to the register number, as shown in the example earlier – or by using their “name”.

Many of the registers are reserved for a special purpose and should not generally be modified by the application programmer.

Number	Name	Purpose
0	\$zero	Always 0. Writes to this register are ignored.
1	\$at	Reserved by the assembler.
2–3	\$v0–\$v1	Expression and function result values.

Number	Name	Purpose
4–7	\$a0–\$a3	Arguments for function calls.
8–15	\$t0–\$t7	Temporary values.
16–23	\$s0–\$s7	These values must be saved before being accessed by the called function.
24–25	\$t8–\$t9	Temporary values.
26–27	\$k0–\$k1	Reserved for use by the operating system.
28	\$gp	Global pointer – points to the middle of the 64K memory block in the static data segment.
29	\$sp	Stack pointer.
30	\$s8	Saved value / frame pointer. Must be saved before being used.
31	\$ra	Return address. Must be saved before being used.

Registers T0–T9 are usually the simplest ones to use for beginners as there are no special requirements concerning their use. Registers S0–S8 must be saved before they are changed by a subroutine. Typically, you should push the existing values in these registers to the stack at the start of a routine, and pop them back from the stack before returning.

Two different instructions are used to move values between registers and memory. The *load* instruction copies the contents of a memory cell into the specified register. Instead of specifying a memory location directly, you can use the name of a variable.

The load instruction has a few different forms:

lb \$register, memory_location

Loads a byte from the memory location and stores it in the register.

lw \$register, memory_location

Assembly Language students also learn



Loads a 32-bit word from the memory location and stores it in the register.

li \$register, value

Loads the value specified with the instruction into a register. This operation uses immediate addressing.

When storing values in memory, a different set of instructions is needed. *Store* copies the contents of a register into the memory location that is stated in the source code. As with loads, there are different forms depending on whether you are working with a byte value or a full word:

sb \$register, memory_location

Store the byte value of the register into the memory cell at the given location.

sw \$register, memory_location

Store the 32-bit word value of the register into the memory cell at the given location.

Addressing Modes

The *li* instruction above is an example of immediate addressing. Two other addressing modes are supported in MIPS assembly language: indirect addressing, and indexed addressing.

Indirect addressing is similar to using a pointer in languages such as C and C++. Instead of accessing the value stored in the specified memory cell, indirect addressing loads the value from the memory address that is stored in a register. For example:

Empower your team. Lead the industry.

Get a subscription to a library of online courses and digital learning tools for your organization with Udemy Business.

Request a demo

```
#
# Udemy.com
# MIPS Programming in Assembly Language
#
        .data
mem1:   .word 0      # declare a 32-bit word to hold an address.
var2:   .byte 8      # declare a byte value.
        .text
la $t0, var2  # load the address of var2 into a temporary register.
sw $t0, mem1  # stores the address in the variable mem1.
lw $t0, mem1  # (not needed here but is shown for clarity)
           # loads the address from variable mem1 into $t0.
lw $t1, ($t0) # loads the value from the address specified by
           # $t0 (which contains the address of var2) into $t1.
```

The instruction *la* is used when loading the address of a variable into a processor register. Note that it is not used when the address is reloaded by *lw \$t0, mem1* because you do not want the address of *mem1*, you want the address stored *in* *mem1*. The example above eventually loads the value 8 into *\$t1*.

Indexed addressing is used to specify an offset from a particular memory address, and is commonly used when working with arrays. The register containing the base address is wrapped in parenthesis as before. However, the number immediately preceding the parenthesis is added to the memory address. For example:

```
#
# Udemy.com
# MIPS Programming in Assembly Language
#
        .data
var1:   .byte 1
var2:   .byte 2
        .text
la $t0, var1  # load the address of var1 into a temporary register.
lb $t1, 1($t0) # loads the value from the address in $t0+1.
```

The code above loads the value 2 into *\$t1*. *1(\$t0)* moves to the byte after the starting address in *\$t0*, and so the memory address used is now the same as the one used for *var2*.

Basic Arithmetic

Math instructions in MIPS are slightly different from those used on other processors, as most of the instructions require three “operands” (arguments). One of these specifies the register in which to store the result.

```
add $t0,$t1,$t2    # is the equivalent of $t0 = $t1 + $t2
```

The instructions that behave this way are:

- `add` – adds two signed integers.
- `addi` – adds an immediate value to a register.
- `addu` – adds two unsigned integers.
- `sub` – subtracts one integer from another.
- `subu` – subtracts one unsigned integer from another.

Multiplication (*mult*) and division (*div*) are special cases because they must return a 64-bit value (in 32-bit MIPS) to ensure no loss of data. The result of the multiplication and division operations is stored in two special variables, *hi* and *lo*, and these can then be moved to other registers using the instructions *mfhi* and *mflo*. Both of these instructions take one operand: the name of the destination register.

Comparisons and Jumps

Making comparisons, and jumping to other parts of a program based on those comparisons, is known as *branching*. MIPS assembly language is unusual in that evaluating conditions is built-in to the instructions that perform the jump.

To make an unconditional jump to a defined label, you can use the instruction:

```
b target_label
```

The other branch instructions follow the same syntax as the *beq* instruction below:

```
beq $register, $register, target_label
```

beq branches to the `target_label` if the values stored in the two registers are equal.

The other comparison instructions are:

- `bne` – branch if the two registers are not equal.
- `blt` – branch if the first register is less than the second register.
- `ble` – branch if the first register is less than or equal to the second register.
- `bgt` – branch if the first register is greater than the second register.
- `bge` – branch if the first register is greater than or equal to the second register.

Further Reading

There are a few notable absences from this introduction to assembly language on MIPS microprocessors. Before learning about subroutines, functions, syscalls, and using the stack, you should ensure that you have a good understanding of the basics introduced so far. You may also want to review how numbers are stored in [binary in computer systems and converting from decimal, hexadecimal, and binary representations](#).

Page Last Updated: June 2014

Recommended Articles

How to Learn Assembly Language



Udemy Editor

Learn Assembly Language: Speak the Language of Your



Udemy Editor



Teach the World Online

Create an online video course, reach students across the globe, and earn money.

Teach on Udemy

Follow us

Terms

Privacy Policy and Cookie Policy

Sitemap

Featured Courses

Get the app

Cookie settings



© 2023 Udemy, Inc.